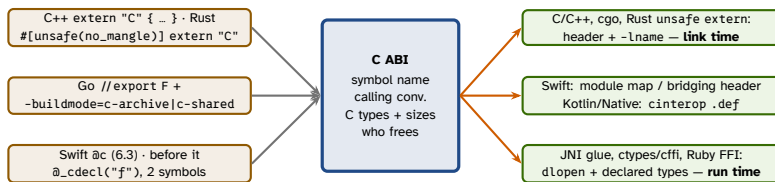


In one line: Languages call each other through the **C ABI**: an unmangled symbol, the platform's C calling convention, C types (scalars, pointers, plain structs). One side **exports** C-looking functions (extern "C", Go //export, Swift @c); the other **links** them (-l, cgo, module map) or **dlopens** them and declares the types (ctypes, Ruby FFI). Every allocation keeps *one* owner.

1 exporters → C ABI → consumers



does not cross: classes, exceptions/panics, generics, GC'd objects, Swift/Go/Java strings

2 why extern "C"

```

void PrintHello(const char *u);
compiled as C++ (Itanium ABI) the symbol is
_Z10PrintHelloPKc
_Z · 10+name · P K c = ptr to const char
— overloads need it; C, cgo, dlsym can't guess it.
inside extern "C": C linkage, plain name
PrintHello (Mach-O: _PrintHello; dlsym without _)
JNI's own scheme — the name is the binding:
Java_com_ex_Lib_hello
Java_ + class + _ + method · _ → _1 · overload: __+signature
see it: nm libexample.a · demangle: c++filt
  
```

Pick a strategy

host · library	strategy (e.g.)
compiled · compiled	link a library via the C ABI (Go → C++, Rust ↔ C)
compiled · interpreted	embed the interpreter, run scripts (CPython C API, Lua)
VM · compiled	FFI or a native extension (ctypes, cffi, JNI)
interpreted · interpreted	2 processes + IPC (pipes, sockets, gRPC) or one C lib
any → Wasm	only i32 i64 f32 f64 cross; a string = ptr + len

Libraries and the linker

- Static** .a = archive of .o (ar rcs); the linker copies in only members that resolve an undefined symbol. GNU ld searches each archive **once, where it stands** (-lfoo *after* its users); LLD remembers archives, so order does not bite there.
- Dynamic** .so/.dylib/.dll (cc -shared -fPIC; macOS -dynamiclib): the loader maps + binds it at launch; dlopen+dlsym at run time = what ctypes/FFI do.
- Both in one dir, -lexample takes the .so/.dylib (GNU ld, ld64): "static" links, then fails to launch elsewhere — use -Bstatic or the .a path. Module map link "example" = -lexample.

Example — one header for C, C++ and cgo

```

#ifdef __cplusplus
extern "C" {           // C++ sees C linkage; C never sees this
#endif
void PrintHello(const char *u);           // C types only
char *MakeGreeting(const char *name);     // malloc'd ...
void FreeGreeting(char *p);               // ... freed HERE
#ifdef __cplusplus
}
#endif
  
```

```

// #cgo LDFLAGS: -L${SRCDIR} -lexample
// #include <stdlib.h>
// #include "example.h"
import "C"           // the comment right above is the preamble
import "unsafe"
func main() {
    s := C.CString("Hi")           // malloc'd C copy
    defer C.free(unsafe.Pointer(s)) // Go frees it
    C.PrintHello(s)
}
  
```

Go as the library

```

//export SayHello
func SayHello(s *C.char) { fmt.Println(C.GoString(s)) }
func main() {} // package main required; never called
// go build -buildmode=c-shared -o libhello.so (c-archive: .a)
  
```

Only //exported functions are callable; the Go runtime ships inside and starts as a global constructor (c-archive) or when the .so loads; libhello.h is written beside it. With //export the preamble may only *declare* (it is copied into two C files).

Python and Ruby — load at run time

```

lib = CDLL("./libexample.so")           # dlopen
lib.PrintHello.argtypes = [C_char_p]    # now type-checked
lib.PrintHello(b"Hi")                   # bytes -> char*
lib.MakeGreeting.restype = c_void_p      # default int truncates
p = lib.MakeGreeting(b"Ann"); print(c_char_p(p).value) # copy
lib.FreeGreeting(c_void_p(p))           # a bare int goes as C int
  
```

Ruby FFI: attach_function :PrintHello, [:string], :void — return type required. CDLL **releases the GIL** (PyDLL keeps it). cffi cdef() takes declarations (no #include); *ABI mode* dlopens, *API mode* compiles an extension (faster).

JVM — JNI

- Kotlin external fun hello(s: String) in com.ex.Lib + System.loadLibrary("example") (→ libexample.so) bind C glue written against jni.h, named as in 2, args (JNIEnv*, jobject, jstring) — or RegisterNatives in JNI_OnLoad.
- JNIEnv* is **per thread** (a thread C started: AttachCurrentThread); a jobject is a **local ref**, dead after the call (NewGlobalRef); strings come as **modified UTF-8**. No glue: FFM API (JDK 22) + jextract.

Memory ownership and errors

- Whoever allocates, frees** — FreeGreeting, Go C.free, Rust CString::from_raw (never C free()).
- C keeps no Go pointer past the call unless pinned (runtime.Pinner); a ctypes CFUNCTYPE must stay referenced while C may call it.
- Errors:** catch in the wrapper, return a code — a Rust panic out of extern "C" aborts (1.81+), a C++ exception reaching Swift terminates.

Traps

- No extern "C"** → *undefined reference to PrintHello*; an unguarded extern "C" in a header breaks C/cgo.
- ctypes:** no restype truncates a 64-bit pointer to int; no argtypes: a str goes as wchar_t* (C prints "H"); restype = c_char_p returns a bytes copy — no pointer left to free.
- C types differ:** long 4 B on Windows x64, 8 B on Linux/macOS; plain char unsigned on Linux/Android arm64; Apple arm64 puts variadic args on the stack (ctypes: set argtypes).

Remember

Speak C at the border: plain names, C types, one owner.